

ENGINEERING DOSSIER · THE
COMPLETE REPOSITORY PICTURE

KEJA AI

The full engineering picture of gadda00/keja-ai:
architecture, the nine-product module map, the Auto-
Pilot data pipeline, PWA internals, quality gates and
deployment — one repository, three runtimes, zero
unanswered questions.

Prepared by the Keja AI engineering team

GADDA00/KEJA-AI · MAIN @ AB30A10

10 SEPTEMBER 2026

KEJA AI · KEJA.APP

Table of Contents

1 Executive Overview	2
2 Architecture and Core Design Decisions	3
One bundle, three runtimes	3
Hash routing and the static export	3
State, storage and the data spine	3
The backend graduation path	4
3 The Module Map	4
Ecosystem products	4
Route inventory	5
Support libraries	6
4 Data and Intelligence Layer	6
The Auto-Pilot ingestion pipeline	6
The KEJA Trust Score and Property Passport	6
AI advisor and Deal Analyst	7
Market intelligence with declared sourcing	7
5 Progressive Web App Implementation	7
Manifest and rich install	7
Service worker strategy	8
iOS Safari specifics	8
6 Native Shells (Capacitor)	8
7 Quality Pipeline	9
Post-mortem: why every deploy failed in early September	9
Why API-zip deploys need <code>_redirects</code> and <code>_headers</code>	10
8 Deployment, Domain and Infrastructure	10
9 Repository Layout	11
10 Development Guide	11
11 Risk Register and Technical Debt	12

1 Executive Overview

This dossier is the complete engineering picture of **gadda00/keja-ai**—the single repository that builds, tests, deploys and operates Keja AI, Africa's real-estate intelligence and trust infrastructure. It documents what exists today, how every layer fits together, and where the honest boundaries between live capability, pilot, trial and plan actually sit. The repository is not a prototype pile: it is a production pipeline that ships to a global CDN on every commit, grows its own inventory on a schedule, and installs as a full-screen app on Android and iOS.

Keja AI is a Chacadom Investments venture with one canonical deployment: **<https://keja.app>**, served from Netlify's edge network, with the working URL <https://keja-ai.netlify.app> live and verified as of 10 September 2026. The product surface is deliberately wide—nine products across discovery, verification, analysis, finance, investment, transactions, tokenization and management, plus four stakeholder portals—because the underlying thesis is that trust infrastructure only compounds when it spans the whole transaction lifecycle. Everything a user sees is generated from this one repository, and every claim the site makes about its own capabilities is anchored to a public claims register that separates live features from simulated ones.

134 TypeScript source files in src/	27,363 Lines of app code	91 React components	30 Routed views
--	------------------------------------	-------------------------------	---------------------------

The stack is Next.js 16.1.3 on React 19 with TypeScript in strict mode, Tailwind CSS 4, the Radix UI primitive set, framer-motion for interaction design and Recharts for data visualisation. The application compiles to a fully static bundle: there is no server runtime in production today, which makes the deployment essentially free to operate, trivially cacheable at the edge, and directly reusable inside the Capacitor Android and iOS shells that wrap the same bundle for app-store distribution. A Prisma-backed development database and API routes exist in the sandbox configuration, ready for the backend graduation, but nothing in the current production path depends on them.

Quality is enforced before anything reaches the public: three GitHub Actions workflows run typecheck, lint and a full static build on every pull request, and the production deploy repeats the same gates before uploading. The result of that discipline is visible in the deploy history—after a dependency-installation fix documented in Chapter 7, the pipeline shipped commit **ab30a10** green end-to-end, with the service-worker cache version stamped **v43c8cafcbc2d** and security headers verified live on the edge.

2 Architecture and Core Design Decisions

The architecture is a deliberate bet: a client-side single-page application with hash routing, statically exported, with zero server dependency in production. This chapter explains why that bet was made, what it buys, and where the seams are for the future backend.

One bundle, three runtimes

The same static export serves three destinations without forking. First, the web: *out/index.html* and its hashed assets are uploaded to Netlify and served from the CDN with immutable caching on */assets/*. Second, the installed PWA: the manifest, service worker and offline shell turn the same bundle into a full-screen, offline-capable app on Android and iOS home screens. Third, the native shells: Capacitor wraps the identical *out/* directory inside Android and iOS projects, so the web product and the store product can never drift apart. This removes the classic failure mode where the app-store build lags the web build by weeks.

Hash routing and the static export

Routing is custom-built in *src/lib/router.tsx* as a `HashRouter`—URLs look like `https://keja.app/#/properties/KIL-001`. Hash routing was chosen because it makes the application a pure static artefact: every deep link resolves to the same *index.html*, so no server rewrites are strictly required and the bundle works from any dumb file host, a CDN, or a device-local Capacitor server. The trade-off is cosmetic (the `#` in the URL) and SEO-neutral at this stage because discoverable content is additionally exposed through prerendered metadata, sitemap and robots artefacts generated by *scripts/generate-sitemap.mjs*. The Netlify redirect rule `/* → /index.html` still exists so that legacy path-style URLs degrade gracefully into the shell rather than 404.

State, storage and the data spine

All user state—accounts, saved searches, portfolios, token positions, landlord ledgers, tenant records—lives in typed stores (a Zustand-based architecture in *src/lib/*Store.ts*) persisted to `localStorage` on the client. The inventory spine is compile-time: *src/data/properties.ts* seeds the marketplace while *src/data/auto-listings.json* carries the Auto-Pilot pipeline's continuously ingested stock (60 live listings across 40 recorded runs at the time of writing). This means the whole product, including its data, is versioned in git, reviewable in pull requests, and reproducible from a clean checkout—an unusually strong integrity property for a marketplace, bought at the cost of rebuild-to-update data semantics that the backend graduation will replace.

The backend graduation path

Nothing in the current design blocks evolution. The repository already carries Prisma with User and Post models and a local SQLite database in the sandbox; the deployment configuration is a one-line change from static export to the standalone Next.js server output (the package.json build script already knows both modes via the NEXT_STATIC flag); and the client stores are isolated behind library boundaries rather than being scattered through components. The intended sequence is: keep the static PWA as the consumer surface, introduce API routes for identity and transactions first, then move the inventory spine behind a real database while the Auto-Pilot continues to feed it.

3 The Module Map

The product decomposes into nine named products and four portals, each mapped to routes in the SPA switch and to feature components under src/components. The tables in this chapter are the authoritative map from user-facing capability to code.

Ecosystem products

Product	What it does	Status
Keja Home	Property discovery and listings—search, facets, comparison, saved-search alerts	Live
Keja Verify	Verification, the KEJA Trust Score (12 labelled factors) and the Property Passport	Live
Keja AI	AI property advisor (EN/SW/FR) plus the AI Deal Analyst with document pre-screening	Live
Keja Invest	Investment calculator, Investment Score and investor dashboard with report export	Live
Keja Data	Market intelligence—natural-language questions answered from live inventory	Live
Keja Finance	Mortgage calculator, CBK 33% DTI eligibility, bank comparison, financing types	Pilot
Keja Transact	Transaction milestones, stakeholder payments and the professional panel	Pilot
Keja Manage	Property and rental management—rent, tenants, maintenance, rule-based AI alerts	Pilot
Keja Token	Fractional ownership—trial mode with fictional assets and a virtual wallet	Trial

Table 3.1 — The nine ecosystem products and their declared maturity.

Route inventory

Thirty routed views are registered in the application shell (`src/components/shell/KejaApp.tsx`), each lazy-loaded so the initial bundle stays lean and heavy workspaces load on demand. The mapping below is exhaustive.

Route	View	Route	View
/	Home	/tokenize	Tokenize (trial)
/properties	Property search	/manage	Manage (landlord)
/properties/:id	Property detail + Passport	/tenant	Tenant Hub
/compare	Compare view	/diaspora	Diaspora Hub
/sell	List a property	/develop	Developer Portal
/ask	Ask Keja AI	/institutional	Institutional Portal
/deal-analyst	AI Deal Analyst	/partners	Partners
/invest	Investment calculator	/trust	Trust Center + claims
/portfolio	Investor dashboard	/ecosystem	Ecosystem overview
/data	Keja Data (market intel)	/insights	Market Insights
/finance	Finance (mortgages)	/insights/:slug	Article detail
/transact	Transact (escrow flow)	/areas/:slug	Area guides
/account	Account	/about	About
/admin	Admin console	/contact	Contact
/pro	Pro workspace	/legal	Terms and Privacy
/valuation	Valuation desk	fallback	In-app 404

Table 3.2 — All 30 routed views in the SPA switch.

Support libraries

Thirty-four modules under `src/lib` implement the domain logic behind those views. The notable ones include `trustScore.ts` (the 12-factor scoring engine), `verification.ts` (evidence and freshness rules), `investmentScore.ts` and `valuationStore.ts` (analytics), `dealAnalyst.ts` (document pre-screening), `marketIntel.ts` (natural-language answering over inventory), `finance.ts` (mortgage and DTI mathematics), `tokenizeStore.tsx` (the simulated ledger), `landlordStore.ts` and `tenantStore.ts` (management domains), `autoListings.ts` (pipeline client), `auth.tsx` (local accounts), `i18n.tsx` (English, Swahili and French dictionaries) and `router.tsx` (the hash router itself). Keeping domain logic in plain, testable modules rather than inside components is what makes the wide feature surface maintainable by a small team.

4 Data and Intelligence Layer

Keja AI's differentiator is not listing volume—it is the trust layer wrapped around every listing, and a self-growing inventory pipeline that runs without human intervention. Both are documented here in detail.

The Auto-Pilot ingestion pipeline

Every six hours (04:10, 10:10, 16:10, 22:10 UTC) the Auto-Pilot workflow wakes up on GitHub Actions and runs `scripts/auto-listings/run.mjs`—a zero-dependency Node pipeline that scans market sources and partner feeds, enriches candidates with AI-generated structured data, deduplicates against existing inventory, applies a quality gate, and publishes the survivors into `src/data/auto-listings.json`. When anything new lands, the bot commits as `keja-autopilot[bot]` and the push itself triggers the production deploy. The marketplace therefore grows end-to-end without a human touching the repository: scan, enrich, gate, publish, deploy. Forty runs are recorded with 60 live auto-listings currently in inventory, and runs that ingest nothing exit with a dedicated code so CI stays green on quiet days.

The KEJA Trust Score and Property Passport

Every listing carries a Trust Score computed from twelve labelled factors in `src/lib/trustScore.ts`—ownership verification, title status, encumbrances, rates clearance, zoning conformity, valuation alignment, listing freshness, evidence completeness and more—each individually inspectable rather than hidden inside an opaque number. The Property Passport extends this into a durable identity document per property (KEJA-XXX-000000 identifiers) that consolidates verification status, valuation band and fraud risk with evidence panels whose freshness is tracked on a 90-day window. The design rule enforced across the codebase is that scores are always explainable: a user can drill from any number to the factors that produced it.

AI advisor and Deal Analyst

The conversational advisor (Ask Keja) is trilingual—English, Swahili, French—with intent routing that hands users off to the right workspace, and a professional-advice escalation guard that sends legal, tax, valuation and suitability questions to humans rather than improvising. The Deal Analyst accepts a described deal plus documents that are processed on-device and never uploaded, then returns an investment score, market-value comparison, yield analysis, red flags and one of four recommendations: Proceed, Negotiate, Investigate or High-Risk. The claims register at /trust declares which parts of these experiences are live versus simulated, and the site is architecturally prevented from claiming more than the register states.

Market intelligence with declared sourcing

Keja Data answers natural-language questions such as which neighbourhood has the highest yield or where a KSh 20M budget works hardest, computed live from actual inventory rather than cached marketing copy, with sample sizes and sourcing declared next to every answer. This honesty-by-construction—answers that show their sample size—is a small implementation detail that quietly separates the product from portals that publish stale, unsourced area statistics.

5 Progressive Web App Implementation

Installability on both Android and iOS was hardened in September 2026 and verified against the deployed site. This chapter records exactly what ships, because PWA details are where most real-estate web apps quietly fail.

Manifest and rich install

The web app manifest (public/manifest.webmanifest) declares standalone display with a minimal-ui fallback, portrait orientation, the brand palette (background #F7F5EF, theme #0E7A4E), and a complete icon set: 192px and 512px PNGs for general use plus dedicated maskable variants with safe-zone padding so adaptive launchers on Android never crop the mark. It also ships app shortcuts (Properties, Ask Keja, Deal Analyst) for long-press menus, and two stylised screenshots—narrow 1080x2340 and wide 1920x1080—which Chromium's rich install dialog displays before installation. `launch_handler` is set to navigate-existing so a second launch focuses the running instance instead of spawning a duplicate window.

Service worker strategy

public/sw.js implements three caches—assets, images and pages—keyed by a version stamped mechanically at deploy time by scripts/sw-version.mjs from a SHA-256 hash of the entire deployed tree. Hashed assets are cache-first and immutable; images are cache-first with 200-only admission; navigations are network-first with an 8-second timeout racing the network, falling back to the cached page, then the shell, then offline.html—a design that fixes the stuck-blank-page failure mode on lie-fi connections common in the target market. Because the cache version is content-derived, any real content change automatically evicts stale caches on the next activate; no-content deploys keep the same version and avoid needless cache churn.

iOS Safari specifics

iOS does not fire beforeinstallprompt, so the InstallPrompt component (src/components/shell/InstallPrompt.tsx) detects Safari on iPhone and iPad—including iPadOS 13+ masquerading as desktop Safari—and renders a three-step walkthrough (Share, Add to Home Screen, Add) instead of a disabled button. Seventeen device-class launch screens in public/splash/ cover current iPhone and iPad classes so a cold start shows the branded green splash rather than a white flash; they are generated deterministically by scripts/generate-pwa-assets.mjs (sharp, zero new dependencies). The document head carries apple-mobile-web-app-capable in both prefixed and unprefixed forms, a black-translucent status bar matched by safe-area padding in the shell, and an apple-touch-icon. Dismissal of the install banner is remembered for 14 days, and already-installed runs (display-mode standalone) or Capacitor shells never see it.

Verification, 10 September 2026: manifest.webmanifest serves from the edge; sw.js reports cache version v43c8cafcbc2d; X-Frame-Options DENY and X-Content-Type-Options nosniff are live on <https://keja-ai.netlify.app>—the API-zip header pipeline is confirmed working in production.

6 Native Shells (Capacitor)

The repository carries first-class Android and iOS projects, not an afterthought: Capacitor 8.5.1 wraps the identical static bundle that serves the web.

The android/ and ios/ directories are complete Gradle and Xcode projects under capacitor.config.ts, with application id com.chacadom.keja, splash-screen and status-bar plugins configured, and the web directory pointed at out/. The npm scripts wire the flow: mobile:sync builds the static export and syncs it into both projects, mobile:android and mobile:ios open the IDEs, and mobile:apk produces a debug APK straight from Gradle. Because the shells load the same bundle as the web PWA, every deploy automatically refreshes the app content on next launch—the store builds only need republishing when native plugins change. The @capacitor/core, cli, android and ios packages plus the splash-screen and status-bar plugins were added to

the manifest in September 2026, fixing a latent gap where the native projects referenced packages that were never declared, breaking typecheck.

7 Quality Pipeline

Three workflows guard the repository. This chapter documents each, and includes the honest post-mortem of the September 2026 deploy failure and its fix, because that incident shaped the current design.

Workflow	Trigger	What it does
PR check	pull requests, non-main pushes	Typecheck, lint, static build—nothing merges red
Deploy to Netlify (keja.app)	pushes to main, manual	Same gates, then zip out/ and deploy via the Netlify API
Auto-Pilot listings	cron every 6h, manual	Ingest pipeline; commits new listings which re-trigger the deploy

Table 7.1 — The three GitHub Actions workflows.

Post-mortem: why every deploy failed in early September

Symptom: the Deploy to Netlify workflow failed on every run with all jobs red, while the Auto-Pilot ingest job kept succeeding. Root cause: all three workflows installed dependencies with npm ci, but the repository's lockfile is bun.lock—there is no package-lock.json—so npm ci exited immediately with nothing to install from. The ingest job never noticed because its pipeline is deliberately zero-dependency Node. The fix, shipped in ab30a10, switched all workflows to oven-sh/setup-bun@v2 with bun install --frozen-lockfile against the lockfile that actually exists, added an explicit guard that fails with instructions if the Netlify credentials are missing, and added a verification step that out/_redirects and out/_headers exist in the bundle. The same commit restored the Capacitor packages that typecheck needed. The run on ab30a10 then completed green: build, stamp, upload and live verification.

Why API-zip deploys need `_redirects` and `_headers`

Netlify only reads `netlify.toml` when it builds from a git connection. The production deploy uploads a zip of `out/` through the API, so the redirect and header rules must ship inside the bundle itself—`public/_redirects` and `public/_headers` are copied into `out/` by the static export and mirror `netlify.toml` exactly. This was a latent gap until September 2026: the SPA fallback and security headers silently did not apply to production. Both files now carry comments binding them to `netlify.toml`, and the workflow verifies their presence before uploading.

8 Deployment, Domain and Infrastructure

Production is a Netlify site fed by GitHub Actions. The domain `keja.app` is registered at Spaceship and already pointed at Netlify's load balancer; the remaining step is resolving a domain-ownership conflict on the Netlify side, documented separately.

The deploy job runs the full quality gauntlet, stamps the service-worker version, zips `out/` and POSTs it to the Netlify API using the `NETLIFY_AUTH_TOKEN` and `NETLIFY_SITE_ID` repository secrets, then polls until the deploy is ready. The working URL `https://keja-ai.netlify.app` is live and verified. The canonical domain `keja.app` is registered with Spaceship nameservers (`launch1/launch2.spaceship.net`) and its A record already points to Netlify's apex load balancer `75.2.60.5`, with the `www` CNAME flow documented in the domain setup guide. The open item is administrative rather than technical: Netlify reports the domain is claimed by another team, and a companion document in this suite walks through releasing it. Once released, `keja.app` attaches to the site in minutes with automatic Let's Encrypt HTTPS.

Environment	URL	Status
Production (working URL)	<code>https://keja-ai.netlify.app</code>	Live, auto-deploys from main
Canonical domain	<code>https://keja.app</code>	DNS pointed; pending Netlify team release
Sandbox / development	<code>local next dev on :3000</code>	Running in the build environment
Android debug build	<code>gradlew assembleDebug</code>	Project configured, Capacitor 8.5.1
iOS build	Xcode project in <code>ios/</code>	Project configured, requires macOS

Table 8.1 — Environments and their current status.

9 Repository Layout

The repository is organised so that a new engineer can find any concern in one hop. Top-level directories and their responsibilities:

Path	Contents
src/components	91 React components across 14 feature domains (property, ai, invest, finance, ...)
src/lib	34 domain modules: scoring, verification, stores, router, i18n, analytics
src/data	Inventory spine: properties.ts, auto-listings.json, articles, claims, neighborhoods, tokenize
src/app	Next.js app router entry: layout, page, globals.css
public	Static assets: manifest, sw.js, icons, splash screens, screenshots, _redirects, _headers
scripts	sw-version, generate-sitemap, generate-pwa-assets, auto-listings pipeline, prerender
.github/workflows	deploy-netlify, pr-check, auto-listings
android/, ios/	Capacitor 8.5.1 native shells (com.chacadom.keja)
prisma/, db/	Schema and local SQLite database for the backend graduation path
docs/	DATA_DICTIONARY, MOBILE guide, review actions, CMA documents
netlify.toml	Build definition and header/redirect rules (mirrored in public/)

Table 9.1 — Top-level repository layout.

10 Development Guide

Everything an operator needs day-to-day. Dependencies install with Bun (the lockfile is bun.lock); Node 22 runs the build.

```

bun install --frozen-lockfile      # install exactly the locked tree
npm run dev                       # dev server on :3000
npx tsc --noEmit                  # typecheck (CI gate)
npm run lint                      # eslint (CI gate)
npm run build:static              # NEXT_STATIC=1 next build -> out/
node scripts/sw-version.mjs       # stamp SW cache version
node scripts/generate-pwa-assets  # regenerate icons/splash/screenshots
npm run mobile:sync               # sync out/ into android + ios
npm run mobile:apk                # debug APK via Gradle

```

The quality bar before pushing: typecheck, lint and a clean static build locally—the same three gates CI enforces. Netlify credentials live as repository secrets and are never needed locally. The .env file in the

sandbox carries only the local database URL and is gitignored; production has no server-side configuration at all today, which also means there are no production secrets to rotate.

11 Risk Register and Technical Debt

An honest dossier names its debts. These are the known limitations, ranked by how much they matter to the product thesis.

Risk / debt	Reality	Path forward
Compile-time inventory	Listings update via bot commits and deploys, not live writes	Move spine behind API + database (Prisma groundwork exists)
Client-only persistence	localStorage means no cross-device accounts yet	Introduce auth API and server-side profiles in the backend phase
Tokenization is trial mode	Fictional assets, virtual wallet, no real securities	Regulatory work with CMA before any real issuance
Domain attachment pending	keja.app still shows Netlify team-conflict	Release via Netlify support (see companion guide)
Single-maintainer bus factor	One main branch, one deployer	Documented dossier, CI gates and this repo picture reduce key-person risk
Analytics depth	Local analytics module, no product analytics SaaS wired	Instrument events before growth spend

Table 11.1 — Known limitations and their remediation paths.

None of these debts are hidden: the in-product claims register states them to users in exactly the same terms. That alignment—the marketing can never outrun the engineering, because the product itself refuses to claim what the register does not declare—is the most valuable quality control the repository has, and this dossier is its paper counterpart.